



ساختمان داده‌ها و الگوریتم‌ها

پاسخ تمرین اول - پیچیدگی و الگوریتم‌های بازگشتی

آریا عازم، مهدی یاری

۲۵ نمره

۱. پیچیدگی زمانی

پیچیدگی زمانی کدهای زیر را بررسی کنید. (در صورت تفاوت بین بهترین و بدترین حالت، هر دو را بیان کنید.)

```
a) int func1 (int n) {  
    int cnt = 0;  
    for (int i = 1; i < n; i *= 2) {  
        for (int j = 0; j < n - i; j++) {  
            cnt++;  
        }  
    }  
    return cnt;  
}  
  
b) void func2 (int n) {  
    int i = n;  
    while (i > 0) {  
        int j = 1;  
        while (j < i) {  
            j *= 2;  
        }  
        i--;  
    }  
}  
  
c) for (int i = 0; i < n; i++)  
    for (int j = 0; j <= i; j++)  
        for (int k = 0; k < n - j; k++){  
            sum += k;  
        }  
  
d) mergeCheck(A, B, m, n) {  
    int i = 0, j = 0;  
    while (i < m && j < n) {  
        if (A[i] == B[j])  
            i++, j++;  
        else if (A[i] < B[j])  
            i++;  
        else  
            j++;  
    }  
    return (i + j);  
}
```

```

    }

e) int func5(int n) {
    int x = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j += 2) {
            x += (i + j);
        }
    }
    return x;
}
    
```

پاسخ:

(a) حلقه بیرونی $\log n$ بار اجرا می‌شود و حلقه داخلی در هر تکرار به اندازه $(n - i)$ بار اجرا می‌شود. بنابراین:

$$O((n - 1) + (n - 2) + \dots + (n/2)) = O(n \log n)$$

(b) حلقه بیرونی n بار اجرا می‌شود. در هر بار اجرای حلقه بیرونی، حلقه داخلی تقریباً $\log(i)$ بار اجرا می‌شود (زیرا j در هر مرحله دو برابر می‌شود). پس پیچیدگی کلی برابر است با:

$$O(\log(1) + \log(2) + \dots + \log(n)) = O(n \log n)$$

(c) حلقه داخلی در هر مرحله $j - n$ بار اجرا می‌شود. حلقه میانی $i + 1$ بار اجرا می‌شود و حلقه بیرونی n بار. بنابراین:

$$\sum_{i=0}^{n-1} \sum_{j=0}^i (n - j) = O(n^2)$$

(d) در هر مرحله از حلقه یا i یا j (یا هر دو) افزایش می‌یابد. در نتیجه، تعداد اجرای حلقه حداکثر برابر با $m + n$ است. بنابراین پیچیدگی زمانی برابر است با:

$$O(m + n)$$

که در بهترین حالت نیز همین مرتبه را دارد.

(e) حلقه داخلی تقریباً نصف مقادیر j را پیمایش می‌کند (چون j هر بار ۲ تا زیاد می‌شود). در نتیجه:

$$O\left(\frac{n}{2} + \frac{n-1}{2} + \dots + 1\right) = O(n)$$

۲۰. نمره

۲. روابط بازگشتی

پیچیدگی روابط بازگشتی زیر را با استفاده از روش‌های ممکن (قضیه اصلی، درخت بازگشتی، یا تبدیل متغیر) به دست آورید و ایده حل را به صورت واضح توضیح دهید.

a) $T(n) = 3T(n - 1) + O(1)$

b) $T(n) = 16T\left(\frac{n}{4}\right) + n^2$

c) $T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + n$

d) $T(n) = T(\sqrt{n}) + T(n^{1/3}) + O(\log n)$

پاسخ:

(a) با باز کردن بازگشتی (unrolling) می‌بینیم که تعداد زیرمسئله‌ها و ضریب رشد به صورت نمایی است.

$$T(n) = 3T(n-1) + O(1)$$

یک بار باز کردن:

$$T(n) = 3(3T(n-2) + O(1)) + O(1) = 3^2T(n-2) + 3 \cdot O(1) + O(1)$$

به طور کلی پس از k باز شدن داریم:

$$T(n) = 3^k T(n-k) + O(1) \sum_{i=0}^{k-1} 3^i$$

اگر $k = n$ (یا تا رسیدن به پایه کوچک) بگیریم، جمله همگرا نیست و جمله اصلی از مرتبه 3^n می‌شود. بنابراین:

$$T(n) = \Theta(3^n)$$

(b) استفاده از قضیه اصلی (Master Theorem) برای رابطه

$$T(n) = 16T(n/4) + n^2$$

پارامترها عبارت‌اند از $a = 16$, $b = 4$. محاسبه پایه مقایسه:

$$n^{\log_b a} = n^{\log_4 16} = n^2$$

بنابراین $f(n) = n^2$ هم‌ردیف با $n^{\log_b a}$ است (یعنی حالت دوم قضیه اصلی با $k = 0$). طبق حالت دوم:

$$T(n) = \Theta(n^{\log_b a} \log n) = \Theta(n^2 \log n)$$

(c) از درخت بازگشتی استفاده می‌کنیم و مجموع هزینه تمام گره‌ها را محاسبه می‌کنیم. در این رابطه، هزینه هر گره برابر با اندازه مسئله آن (یعنی n در ریشه، و به تبع آن size برای هر گره) است.

$$T(n) = T(n/2) + T(n/4) + n$$

سطح یک: دو گره با اندازه‌ها $n/2$ و $n/4$ که هزینه کل سطح برابر است با

$$\frac{n}{2} + \frac{n}{4} = \frac{3}{4}n.$$

سطح دوم: باز هم مجموع اندازه‌ها برابر با $n \left(\frac{3}{4}\right)^2$ است (زیرا هر زیرشاخه به نسبت $2/1$ و $4/1$ تقسیم می‌شود و جمع سهم‌ها در هر سطح یک ضریب $\frac{3}{4}$ نسبت به سطح قبل دارد). بنابراین هزینه سطح i ام برابر است با $n \left(\frac{3}{4}\right)^i$. چون $\frac{3}{4} < 1$ ، مجموع هندسی سطوح همگراست:

$$T(n) = n \sum_{i \geq 0} \left(\frac{3}{4}\right)^i = n \cdot \frac{1}{1 - \frac{3}{4}} = 4n.$$

در نتیجه:

$$T(n) = \Theta(n)$$

(d) تبدیل متغیر نمایی $n = 2^m$ تا روابطی با مقیاس خطی در m به دست آوریم، سپس درخت بازگشتی روی m بسازیم. فرض کنید $S(m) = T(2^m)$. آنگاه:

$$T(n) = T(n^{1/2}) + T(n^{1/3}) + O(\log n)$$

را با $n = 2^m$ تبدیل می‌کنیم:

$$S(m) = S\left(\frac{m}{2}\right) + S\left(\frac{m}{2}\right) + O(m)$$

حال مانند سوال c مجموع اندازه‌ها در هر سطح را حساب می‌کنیم:

$$\frac{m}{2} + \frac{m}{2} = m$$

سطح یک: $\frac{5}{6}m$ ، و به همین ترتیب ضریب هر سطح $m \left(\frac{5}{6}\right)^i$ خواهد بود. چون $\frac{5}{6} < 1$ ، جمع هندسی سطوح همگراست:

$$S(m) = m \sum_{i \geq 0} \left(\frac{5}{6}\right)^i = \Theta(m).$$

بازگرداندن تغییر متغیر $m = \log n$ نتیجه می‌دهد:

$$T(n) = \Theta(\log n).$$

۳. مرتب‌سازی توابع

۲۰ نمره

با ذکر راه حل، توابع را بر حسب درجه رشد مرتب کنید.

$$7n^2, \log_\delta(n), 50n, n^{3/4}, n^{1/100}, \log(n!), n^n, 5^n, 5^{n+1}, 10^n, 3^n, n \log n, 256n^{3/4}, \log((\log n)^5), 777^{777}, n!, (n-2)!$$

پاسخ:

برای حل به نکات زیر توجه کنید:

- مرتب‌سازی بر اساس رفتار برای $n \rightarrow \infty$ انجام می‌شود.
- $\log_\delta n = \Theta(\log n)$ و $\log((\log n)^5) = 5 \log \log n = \Theta(\log \log n)$.
- برای هر ثابت $\varepsilon > 0$ و هر تابع لگاریتمی داریم $\log n = o(n^\varepsilon)$.
- اگر $0 < a < b$ آنگاه $n^a = o(n^b)$.
- $\log(n!) = \Theta(n \log n)$ (با استفاده از قضیه استرلینگ یا جمع لگاریتم‌ها).
- $5^{n+1} = \Theta(5^n)$ (تنها اختلاف یک ضریب ثابت دارد).
- برای هر ثابت $c > 1$ ، c^n از هر چندجمله‌ای سریع‌تر رشد می‌کند، و در نهایت $n!$ و n^n از همه سریع‌ترند (برای n بزرگ).
- $\log((\log n)^5) = \Theta(\log \log n) \ll \log_\delta n = \Theta(\log n)$.
- $\log_\delta n < n^{1/100}$ پس $\log n = o(n^{1/100})$.
- $n^{3/4} < n^{3/4} < n^{3/4} < n^{3/4}$ و همه این‌ها $o(n)$ برای $n \rightarrow \infty$ نیستند؛ در واقع $n^{3/4} < n$.
- $n^{3/4} = o(n)$ چون $256n^{3/4} = o(50n)$.
- $50n < n \log n \approx \log(n!) = \Theta(n \log n)$ و $50n = o(n \log n)$.
- $7n^2$ بزرگ‌تر از $n \log n$ ولی کوچک‌تر از هر c^n با $c > 1$.
- $3^n < 5^n \approx 5^{n+1} < 10^n$.

- هر c^n برای ثابت c در نهایت از $(n-2)!$ کوچکتر است یا بزرگ‌تر؟ در واقع برای هر ثابت c به ازای n بزرگ، $(n-2)!$ از c^n بزرگ‌تر می‌شود، بنابراین ترتیب به صورت $n! < n^n < (n-2)! < c^n < \dots$ است.

حالا در نتیجه نکات بالا مرتب‌سازی را انجام می‌دهیم. مرتب‌سازی نهایی (از کندترین به سریع‌ترین رشد برای $n \rightarrow \infty$):

$$n \log n \approx \log(n!) < n^2 < 3^n < 5^n \approx 5^{n+1} < 10^n < (n-2)! < n! < n^n$$

- علامت $\approx$ بین 5^n و 5^{n+1} نشان‌دهنده برابری مرتبه (فقط اختلاف ضریب ثابت) است.
- قرار دادن $(n-2)!$ قبل از $n!$ و پس از تمام نماهای ثابت پایه به این دلیل است که فاکتوریل‌ها در بلندمدت از هر c^n بزرگ‌تر می‌شوند، و $(n-2)! < n!$.
- n^n به عنوان بزرگ‌ترین تابع فهرست‌شده، سریع‌ترین رشد را دارد چون هر جمله در حاصل ضرب آن برابر n است که از عوامل $n!$ و از هر c^n بزرگ‌تر است.

۴. تابع بازگشتی عجیب

۱۵ نمره

برای تابع زیر:

```
void weirdo(int n)
{
    if (n <= 1) return;
    int L = floor(log2n);

    for (int k = 1; k <= L; k++) {
        weirdo(n >> k);
    }

    for (int i = 0; i < n; i++)
        do_constant_work();
}
```

الف) رابطه بازگشتی زمان اجرا را بنویسید.

ب) این رابطه را حل کنید و مرتبه زمانی $T(n)$ را به دست آورید.

ج) اگر به جای حلقه دوم، یک $\Theta(1)$ کار انجام می‌شد، چه تغییری در پاسخ ایجاد می‌شد؟

پاسخ:

الف) نوشتن رابطه بازگشتی:

در هر فراخوانی ابتدا برای هر k از ۱ تا $\lfloor \log_2 n \rfloor$ یک فراخوانی بازگشتی روی اندازه $\lfloor n/2^k \rfloor$ انجام می‌شود و پس از حلقه $\Theta(n)$ کار غیربازگشتی انجام می‌شود. بنابراین رابطه:

$$T(n) = \sum_{k=1}^{\lfloor \log_2 n \rfloor} T(\lfloor n/2^k \rfloor) + \Theta(n), \quad T(1) = \Theta(1)$$

ب) حل رابطه و بدست آوردن مرتبه:

ایده کلی: حدس می‌زنیم $T(n) = \Theta(n \log n)$ و با تحلیل درخت بازگشتی یا جانشینی ثابت می‌کنیم. فرض کنیم $L = \lfloor \log_2 n \rfloor$ برای ساده‌سازی از تقریب‌های ثابت صرف نظر می‌کنیم.

فرض القایی: برای هر $m < n$ داشته باشیم $T(m) \leq C m \log m$ برای ثابت مناسب C . اکنون سمت راست را تخمین می‌زنیم:

$$\sum_{k=1}^L T(n/2^k) + \Theta(n) \leq \sum_{k=1}^L C \cdot \frac{n}{2^k} \log\left(\frac{n}{2^k}\right) + c'n$$

که c' ثابت مربوط به جمله $\Theta(n)$ است. این عبارت را باز کنیم:

$$= Cn \sum_{k=1}^L \frac{1}{2^k} (\log n - k) + c'n = Cn \left(\log n \sum_{k=1}^L \frac{1}{2^k} - \sum_{k=1}^L \frac{k}{2^k} \right) + c'n$$

دو جمع معروف را می‌دانیم:

$$\sum_{k=1}^{\infty} 1/2^k = 1, \quad \sum_{k=1}^{\infty} k/2^k = 2$$

برای $L = \lfloor \log_2 n \rfloor$ این مقادیر تا خطای ثابت نزدیک هستند. پس تقریباً داریم:

$$\leq Cn(\log n \cdot 1 - 2) + c'n = Cn \log n - (2C - c')n$$

اگر C را آنقدر بزرگ انتخاب کنیم که $2C - c' \geq 0$ (یعنی $C \geq c'/2$)، آنگاه مجموع بالا کمتر یا مساوی $Cn \log n$ خواهد بود برای همه n بزرگ. بنابراین حکم القایی برقرار است و نتیجه می‌گیریم:

$$T(n) = O(n \log n)$$

برای نشان دادن کران پایین Θ نیز می‌توان حداقل $bound$ بدست آورد: چون در جمع اولین جمله $T(n/2)$ حضور دارد که به نظر $\geq c(n/2) \log(n/2)$ برای ثابت مناسب است و در مجموع منجر به کران پایین $\Omega(n \log n)$ می‌شود. یا به شکل دیگر با حدس $T(n) = \Theta(n \log n)$ و محاسبه مشابه ثابت می‌شود که دو طرف هم مرتبه‌اند. در نتیجه:

$$\boxed{T(n) = \Theta(n \log n)}$$

ج) تغییر در صورت انجام $\Theta(1)$ کار درون حلقه:

اگر به جای انجام $\Theta(n)$ کار پس از حلقه، در هر فراخوان فقط $\Theta(1)$ کار انجام شود، رابطه به شکل زیر تبدیل می‌شود:

$$T(n) = \sum_{k=1}^{\lfloor \log_2 n \rfloor} T(\lfloor n/2^k \rfloor) + \Theta(1).$$

اگر $T(m) = \Theta(m)$ فرض کنیم:

$$RHS \approx \sum_{k=1}^L c \frac{n}{2^k} + O(1) = cn \sum_{k=1}^L \frac{1}{2^k} + O(1) \approx cn$$

که با فرض $T(n) = \Theta(n)$ سازگار است. بنابراین در این حالت:

$$\boxed{T(n) = \Theta(n)}$$

نتیجه گیری: انجام $\Theta(n)$ کار در هر فراخوان درجه را به $\Theta(n \log n)$ می‌رساند، در حالی که اگر تنها $\Theta(1)$ کار انجام شود، مرتبه خطی $\Theta(n)$ حاصل می‌شود.

۵. اثبات یا رد روابط

۱۰ نمره

روابط زیر را با استفاده از تعریف (اثبات مستقیم) یا با مثال نقض رد کنید. در صورت صحیح بودن اثبات ریاضی داده و در غیر این صورت مثال نقض بزنید.

- $O(f \cdot g) = O(f) \cdot O(g)$
- if $g = O(f)$ then $g \cdot n = O(f)$
- if $g = O(f)$ then $g = O(f + n)$
- if $f = \Theta(g)$ and $g = \Theta(h)$ then $f = \Theta(h)$
- if $f = o(g)$ then $g = O(f)$

پاسخ:

$$(a) \text{ ادعا: } O(f \cdot g) = O(f) \cdot O(g)$$

ایده حل: هر تابع در $O(f)$ کران بالایی با f دارد و هر تابع در $O(g)$ کران بالایی با g . حاصل ضرب دو کران بالایی، کران بالایی برای حاصل ضرب توابع است.

فرض کنید $u \in O(f)$ و $v \in O(g)$. آنگاه وجود دارند مقادیر مثبت c_1, c_2, n_1, n_2 طوری که برای همه $n \geq \max(n_1, n_2)$:

$$u(n) \leq c_1 f(n), \quad v(n) \leq c_2 g(n)$$

بنابراین

$$u(n)v(n) \leq c_1 c_2 f(n)g(n)$$

که نشان می‌دهد $u \cdot v \in O(fg)$. این اثبات به ما می‌دهد $O(f) \cdot O(g) \subseteq O(fg)$. برای جهت معکوس، هر تابع $w \in O(fg)$ را نمی‌توان الزاماً به صورت حاصل ضرب دو تابع یکی از $O(f)$ و دیگری از $O(g)$ نوشت (مثلاً $w = fg$ خودش حاصل ضرب است، اما یک $w = \sqrt{fg}$ مانند $w = \sqrt{fg}$ ممکن است قابل تجزیه به $u \in O(f)$ و $v \in O(g)$ نباشد). لذا به طور کلی $O(f) \cdot O(g) = O(fg)$ قابل قبول است به شرط آنکه سمت چپ را به معنی مجموعه حاصل ضرب اعضای مجموعه‌ها تفسیر کنیم؛ در این تعبیر داریم

$$O(f) \cdot O(g) \subseteq O(fg),$$

و هر عنصر از $O(fg)$ لزوماً نماینده حاصل ضرب دو عنصر مناسب از $O(f)$ و $O(g)$ نیست، پس برابری مجموعه‌ای همیشه برقرار نیست. $O(f) \cdot O(g) \subseteq O(fg)$ و برابر بودن دقیق دو مجموعه لزوماً صحیح نیست؛ بنابراین اگر منظور از برابر بودن مجموعه‌ای باشد، این ادعا به طور کلی غیر دقیق است مگر با تعبیر خاص.

$$(b) \text{ ادعا: اگر } g = O(f) \text{ آنگاه } g \cdot n = O(f).$$

ایده حل / مثال نقض: ضرب g در n معمولاً رشد را افزایش می‌دهد؛ بنابراین نمی‌توان انتظار داشت که همیشه هنوز در همان مرتبه f بماند. کافی است یک مثال ساده بزنیم.

مثال نقض: در نظر بگیرید $f(n) = n$ و $g(n) = n$. واضح است $g = O(f)$ (با ثابت $c = 1$). ولی $g \cdot n = n \cdot n = n^2$ در مرتبه $O(n)$ نیست. بنابراین:

$$g \cdot n \notin O(f).$$

نتیجه: ادعا نادرست است و مثال نقض ارائه شد.

$$(c) \text{ ادعا: اگر } g = O(f) \text{ آنگاه } g = O(f + n).$$

ایده حل: اگر $g \leq cf$ برای همه n بزرگ‌تر از n ، آنگاه چون $f + n \geq f$ برای همه $n \geq 1$ ، ضریب ثابت c همان‌جا کار می‌کند و کران برای g نسبت به $f + n$ نیز برقرار است.

اثبات خلاصه: از تعریف $g = O(f)$ وجود دارد c, n ، طوری که برای همه $n \geq n$: $g(n) \leq cf(n)$. چون برای همه n داریم $f(n) \leq f(n) + n$ ، از نامساوی قبلی نتیجه می‌شود

$$g(n) \leq cf(n) \leq c(f(n) + n),$$

بنابراین $g = O(f + n)$.

نتیجه: ادعا صادق است؛ ایده کلیدی این است که $f + n$ حداقل به اندازه f بزرگ است، پس کران‌پذیری نسبت به f کران‌پذیری نسبت به $f + n$ را به همراه دارد.

(d) ادعا: اگر $f = \Theta(g)$ و $g = \Theta(h)$ آنگاه $f = \Theta(h)$.

ایده حل: Θ بیانگر هر دو جهت O و Ω است؛ هرکدام از این روابط به صورت غیرمستقیم قابل ترکیب و استفاده برای اثبات ترانزیتی است. اثبات خلاصه: $f = \Theta(g)$ یعنی وجود ثابت‌های مثبت a, b, n_1 طوری که برای $n \geq n_1$:

$$a g(n) \leq f(n) \leq b g(n).$$

و $g = \Theta(h)$ یعنی وجود c, d, n_2 طوری که برای $n \geq n_2$:

$$c h(n) \leq g(n) \leq d h(n).$$

با ترکیب نامساوی‌ها برای $n \geq \max(n_1, n_2)$ به دست می‌آوریم:

$$a(c h(n)) \leq a g(n) \leq f(n) \leq b g(n) \leq b(d h(n)),$$

یعنی برای ثابت‌های ac و bd داریم $ac h(n) \leq f(n) \leq bd h(n)$. این همان تعریف $f = \Theta(h)$ است. نتیجه: ادعا صادق است؛ ترانزیتی است.

(e) ادعا: اگر $f = o(g)$ آنگاه $g = O(f)$.

ایده حل / مثال نقض: رابطه $f = o(g)$ یعنی f خیلی کوچک‌تر از g است ($\lim_{n \rightarrow \infty} f(n)/g(n) = 0$). این به هیچ وجه نشان نمی‌دهد که g کران بالایی نسبت به f داشته باشد؛ در واقع برعکس، g معمولاً بزرگ‌تر است. یک مثال ساده متضاد کافی است. مثال نقض: بگذارید $f(n) = n$ و $g(n) = n^2$. آنگاه $f = o(g)$ چون $n/n^2 = 1/n \rightarrow 0$ ولی $g = n^2$ لزوماً در $O(f) = O(n)$ نیست، زیرا n^2 را نمی‌توان با ثابت ضرب در n برای همه n بزرگ‌تر کران‌دار کرد. بنابراین $g \notin O(f)$. نتیجه: ادعا نادرست است؛ $f = o(g)$ معمولاً نشان‌دهنده این است که g خیلی بزرگ‌تر از f است، نه اینکه g از f کران‌پذیر باشد.

۶. عملکرد تابع بازگشتی

۱۰ نمره

توضیح دهید تابع زیر چه عملیاتی انجام می‌دهد، سپس رابطه بازگشتی زمان اجرای آن را بنویسید، مرتبه زمانی را محاسبه کنید و تعداد مقایسه‌های عناصر آرایه را (دقیق یا کران بالا) محاسبه و اثبات کنید.

```
typedef struct { long value; int freq; } Res;

Res function(long arr[], int size)
{
    if(size == 1) {
        Res r; r.value = arr[0]; r.freq = 1;
        return r;
    }

    int mid = size / 2;

    Res v1 = function(arr + mid, size - mid);
    Res v2 = function(arr, mid);
```

```

    if (v1.value > v2.value)
        return v1;
    else if (v1.value < v2.value)
        return v2;
    else {
        Res r; r.value = v1.value; r.freq = v1.freq + v2.freq;
        return r;
    }
}

```

پاسخ:

شرح عملکرد: تابع ورودی را به دو بخش تقسیم می‌کند. در واقع تابع به دو نیمه چپ و راست از وسط افراز می‌شود که اشتراکی با هم ندارند. به صورت بازگشتی در هر نیمه بیشینه و فرکانس آن را محاسبه می‌کند. سپس بین دو بیشینه بدست آمده یک مقایسه انجام می‌دهد: - اگر بیشینه نیمه راست بزرگ تر بود، آن را برمی گرداند. - اگر بیشینه نیمه چپ بزرگ تر بود، آن را برمی گرداند. - اگر هر دو برابر بودند، مقدار مشترک و جمع فرکانس‌ها را برمی گرداند.

رابطه بازگشتی زمان: در هر گام تابع دو فراخوان بازگشتی روی دو زیرآرایه با اندازه‌های تقریباً $n/2$ انجام می‌دهد و پس از بازگشت یک جمع یا مقایسه ثابت (چند عمل $O(1)$) انجام می‌دهد. پس برای $T(n)$ داریم:

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + O(1)$$

برای ساده سازی می نویسیم:

$$T(n) = 2T(n/2) + O(1)$$

محاسبه مرتبه زمانی: با استفاده از قضیه اصلی یا تحلیل درخت بازگشتی داریم $f(n) = O(1)$, $b = 2$, $a = 2$. چون $n^{\log_b a} = n^{\log_2 2} = n$ و $O(n^{\log_b a - 1}) = O(n^{\log_2 2 - 1}) = O(1)$ (یعنی خیلی کوچک تر از n)، در واقع این حالت تقسیم "درختی" منجر به جمع سطوحی می‌شود که در مجموع $\Theta(n)$ کار انجام می‌دهد. در نتیجه:

$$T(n) = \Theta(n)$$

تعداد مقایسه‌های عناصر: هر گره داخلی در مرحله ترکیب دقیقاً ۱ مقایسه بین دو مقدار برگردانده شده (مقایسه $v1.value$ و $v2.value$) انجام می‌دهد. برگ‌ها (وقتی $size = 1$) هیچ مقایسه بین عناصر انجام نمی‌دهند. بنابراین اگر تعداد عناصر برابر n باشد، تعداد گره‌های برگ مساوی n و در یک درخت دودویی کامل با n برگ، تعداد گره‌های داخلی دقیقاً $n - 1$ است. پس تعداد مقایسه‌ها:

$$C(n) = n - 1$$